



Systèmes et Réseaux 1

Partie *Systèmes d'exploitation*

3 – Primitives d'Entrées-Sorties Unix

Primitives d'Entrées-Sorties Unix : introduction

- « primitives » ou « appels système » :
 - ◆ services offerts par le noyau du système ;
 - ◆ activés par des appels de fonctions.
- ici on s'intéresse au sous-ensemble « *entrées-sorties de bas niveau* » qui permettent d'interagir avec le système de gestion des fichiers.
- on ne verra que quelques fonctions et quelques paramètres (voir SR2, M1...).

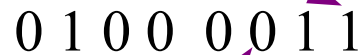
Attention : ne pas confondre les primitives Unix (vues ici) avec les fonctions d'accès aux fichiers de la bibliothèque standard du C (vues en cours de C)

Digression C : Représentation d'un ensemble par un entier

- le type Ensemble n'existe pas en langage C
- pour représenter un ensemble on utilise un entier dont les bits positionnés à 1 dans sa représentation binaire indiquent les éléments présents :

l'élément en position 0 dans la représentation binaire est présent

0 1 0 0 0 0 1 1



l'élément en position 2 est absent

- chaque élément est représenté par une puissance de 2.

$$0000\ 0001 = 2^0 = 1$$

$$0000\ 0010 = 2^1 = 2$$

$$0000\ 0100 = 2^2 = 4$$

Digression C : Représentation d'un ensemble par un entier

Exemple :

- Définition du type

```
typedef unsigned char Soupe;      /* Ensemble pouvant comporter jusqu'à 8 éléments */
```

- Définition des éléments

```
#define PATATES      1      /* 00000001 */
#define CAROTTES     2      /* 00000010 */
#define NAVETS       4      /* 00000100 */
#define OIGNONS      8      /* 00001000 */
#define FEVES       16      /* 00010000 */
#define POIREAUX    32      /* 00100000 */
#define CELERI     64      /* 01000000 */
#define COURGE    128      /* 10000000 */
```

Digression C : Représentation d'un ensemble par un entier

Exemple :

- Déclaration d'un ensemble

Soupe s;

- Construction d'un ensemble :

opérateur | (*ou logique* bit à bit)

```
s=CAROTTES|NAVETS|OIGNONS|POIREAUX;          /* 00101110 */
```

- Détermination de l'appartenance d'un élément à un ensemble :

opérateur & (*et logique* bit à bit)

```
if (s&OIGNONS)
```

```
    printf("Il y a des oignons dans la soupe.\n");
```

- Cette technique est par exemple utilisée au niveau des paramètres de la fonction `open` qui fait partie de la bibliothèque Unix (primitives) de manipulation des fichiers.

Primitives d'Entrées-Sorties : Ouverture d'un fichier

```
#include <sys/types.h>    /* types implémentés différemment selon la version d'Unix */
#include <sys/stat.h>      /* constantes symboliques pour les droits d'accès */
#include <fcntl.h>         /* constantes symboliques pour les différents types d'ouverture */

int open (const char *désignation, int mode)
int open (const char *désignation, int mode, mode_t droits)
int creat (const char *désignation, mode_t droits)
```

- désignation : chaîne de caractères contenant la désignation du fichier.
- mode : entier obtenu par la combinaison de constantes symboliques grâce à l'opérateur « *ou* bit à bit » (`|`) permettant de décrire le mode d'ouverture du fichier.
- Exemple : ouvre le fichier "fichier.txt" en écriture à la fin (*cf diapo suivante*) :

```
int f = open ("fichier.txt", O_WRONLY|O_APPEND) ;
```

Primitives d'Entrées-Sorties : Ouverture d'un fichier

- mode : doit comporter l'une des constantes suivantes :
 - O_RDONLY : ouverture en lecture
 - O_WRONLY : ouverture en écriture
 - O_RDWR : ouverture en lecture/écriture (⚠ $\neq$ O_RDONLY | O_WRONLY)et éventuellement une combinaison des constantes suivantes (*les plus courantes*) :
 - O_APPEND : concaténation des données en fin de fichier
 - O_CREAT : création du fichier lorsqu'il n'existe pas
 - O_EXCL : combiné avec O_CREAT provoque une erreur si le fichier existe (open retourne -1)
 - O_TRUNC : efface le contenu du fichier s'il existe
 - O_SYNC : écriture immédiate sur disque en vidant les tampons (*cf. diapo 16*)

Primitives d'Entrées-Sorties : Ouverture d'un fichier

- droits : si `O_CREAT` est utilisé, un 3^e paramètre indique les droits du fichier créé.

Il est construit grâce à l'opérateur `|` et aux constantes suivantes :

- ◆ `S_IRUSR` : droit de lecture pour le propriétaire
- ◆ `S_IWUSR` : droit d'écriture pour le propriétaire
- ◆ `S_IRGRP` : droit de lecture pour le groupe
- ◆ `S_IWGRP` : droit d'écriture pour le groupe
- ◆ `S_IROTH` : droit de lecture pour les autres
- ◆ `S_IWOTH` : droit d'écriture pour les autres
- ◆ `S_IXUSR` : droit d'exécution pour le propriétaire
- ◆ `S_IXGRP` : droit d'exécution pour le groupe
- ◆ `S_IXOTH` : droit d'exécution pour les autres
- ◆ `S_IRWXU` : droits de lecture, écriture et exécution pour le propriétaire
- ◆ `S_IRWXG` : droits de lecture, écriture et exécution pour le groupe
- ◆ `S_IRWXO` : droits de lecture, écriture et exécution pour les autres

Remarque : `S_IRWXU == S_IRUSR|S_IWUSR|S_IXUSR`

Primitives d'Entrées-Sorties : Ouverture d'un fichier

- `open` retourne le descripteur interne du fichier :
indice (entier naturel) dans la table des descripteurs associée au processus
ou -1 en cas de problème (dépend du mode d'ouverture)*.
- `creat` est équivalente à `open` en mode `O_CREAT|O_WRONLY|O_TRUNC`

Remarque I : l'effet de `O_RDONLY|O_TRUNC` varie selon les implémentations.
Dans plusieurs systèmes, le fichier est en fait tronqué !

Remarque II : les descripteurs de fichiers correspondant aux unités standard** sont les constantes `STDIN_FILENO`, `STDOUT_FILENO` et `STDERR_FILENO` définies dans `unistd.h` et qui valent respectivement 0, 1 et 2. Ces trois unités standard sont automatiquement ouvertes à la création du processus.

* sous Linux, le nombre max de fichiers ouverts dans un processus est 1024 (`getconf -a | grep OPEN_MAX`)

** entrée standard, sortie standard, sortie standard des erreurs (cf. Cours sur le Shell - BAS)

Primitives d'Entrées-Sorties : Fermeture d'un fichier

```
int close(int fd)
```

- ferme le fichier de descripteur fd (en fait, elle libère le descripteur fd) ;
- retourne 0 si la fermeture s'est bien passée ou -1 en cas d'erreur (descripteur incorrect ou disque plein par exemple).

→ à vérifier si ouvert en écriture : possible erreur lors de la fermeture...

Remarque : il est important de fermer un fichier quand on en a fini avec lui. Ainsi le système peut libérer les ressources mémoires associées (descripteur dans la table des descripteurs, tampons mémoire de lecture/écriture,...) et synchroniser les données écrites (pour l'instant dans les tampons en mémoire centrale) sur le disque (*cf. diapo 16*).

Primitives d'Entrées-Sorties : Lecture dans un fichier

```
ssize_t read(int fd, void * p, size_t nboctets)
```

- essaie de lire `nboctets` octets dans le fichier de descripteur `fd` et les range à partir de l'adresse `p`
- retourne le nombre d'octets effectivement lus. S'il est inférieur à `nboctets` alors c'est la fin du fichier.

Remarques : `ssize_t` est un type entier (signé) retourné par la fonction `read`
et `size_t` est un type entier (naturel) retourné par l'opérateur `sizeof`

Par exemple `sizeof(int)` retourne ...

Primitives d'Entrées-Sorties : Lecture dans un fichier

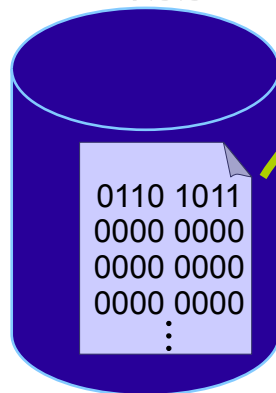
```
ssize_t read(int fd, void * p, size_t nbocets)
```

- Exemple :

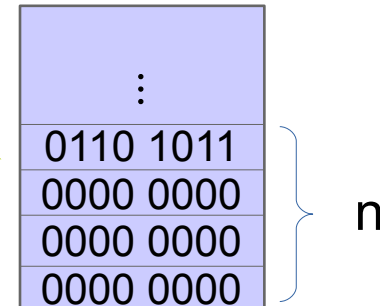
```
int n ;
```

```
read( fd, &n, sizeof(n)) ;
```

HDD/SSD



Pile



n = ?

Primitives d'Entrées-Sorties : Écriture dans un fichier

```
ssize_t write(int fd, const void * p, size_t nboctets)
```

- essaie d'écrire `nboctets` octets à partir de l'adresse `p` dans le fichier de descripteur `fd`
- retourne le nombre d'octets *effectivement* écrits

Remarque : Si le nombre d'octets écrits est inférieur à `nboctets` octets, c'est que le quota de l'utilisateur·ise est atteint ou le disque est plein.

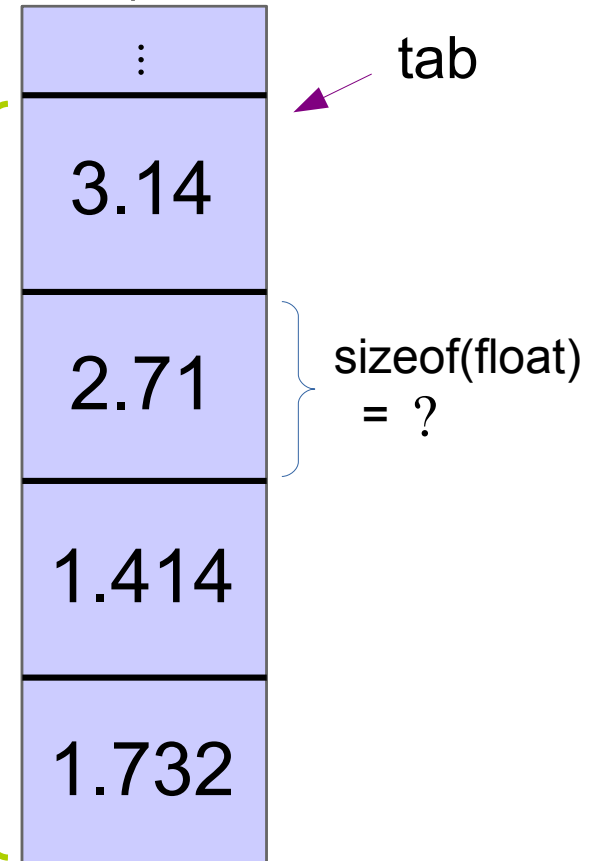
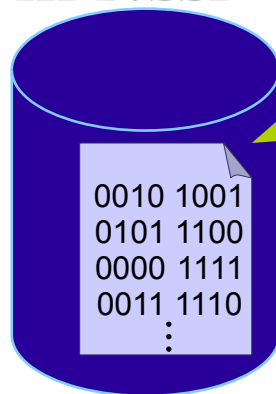
Primitives d'Entrées-Sorties : Écriture dans un fichier

```
ssize_t write(int fd, const void * p, size_t nboctets)
```

- Exemple :

```
float tab[] = { 3.14, 2.71, 1.414, 1.732};  
write( fd, tab, sizeof(tab)) ;
```

HDD/SSD



Primitives d'Entrées-Sorties : Lecture/Écriture d'un fichier

- Exercice 3.1

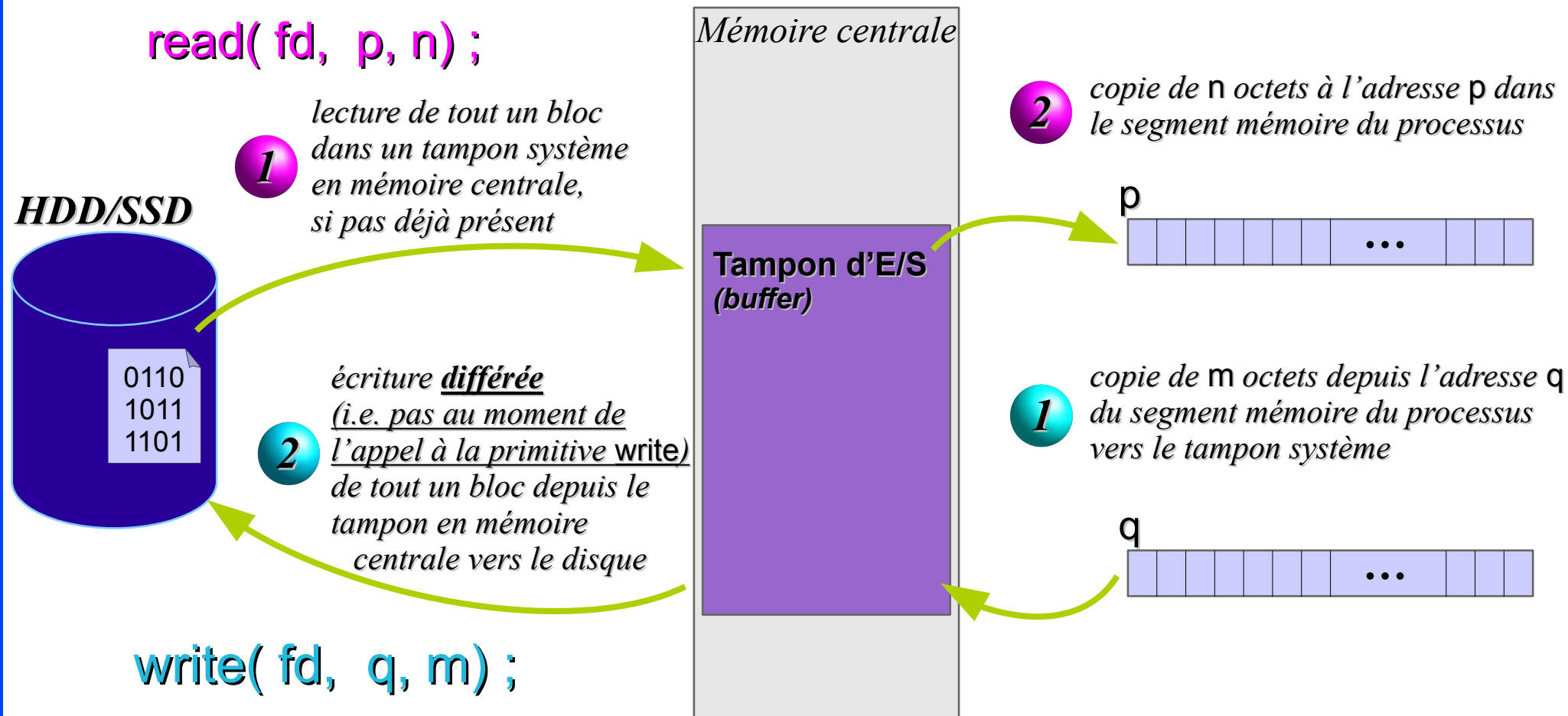
écrire un programme C qui effectue la copie du contenu d'un fichier par blocs de 512 octets.

- Ce programme prend deux arguments : la désignation du fichier source et la désignation du fichier destination.

Si le fichier destination n'existe pas, il doit être créé avec le droit en lecture pour le propriétaire et le groupe, et le droit en écriture pour le propriétaire. S'il existe, son contenu doit être « écrasé ».

Le programme doit gérer les codes de retour en cas d'erreur (mauvais nombre de paramètres, fichier inaccessible, erreurs de lecture-écriture...).

Primitives d'Entrées-Sorties : Tampons E/S en lecture/écriture



Primitives d'Entrées-Sorties : Contrôle de la position courante

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

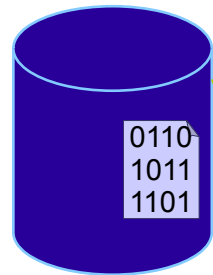
```
off_t lseek(int fd, off_t décalage, int origine)
```

- modifie la position courante du fichier de descripteur `fd`. La nouvelle position est située à `décalage` octets de l'origine qui peut être : `SEEK_SET` (début), `SEEK_CUR` (position courante) ou `SEEK_END` (fin) ;
- retourne la nouvelle position courante exprimée en nombre d'octets depuis le début du fichier ou `-1` en cas d'erreur ;

Remarque : les descripteurs de fichier dupliqués par `fork` ou `dup` (*cf. plus loin*) partagent le même pointeur de position. Ainsi le déplacement sur de tels fichiers peut conduire à des problèmes d'accès concurrents – i.e. pour un fichier ouvert dans le *Père* avant la création du *Fils*, un déplacement dans le *Père* sera effectif pour le *Fils* et vice-versa – *cf. diapo 28*.

Primitives d'Entrées-Sorties : Contrôle de la position courante

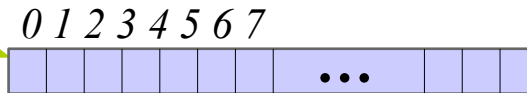
HDD/SSD



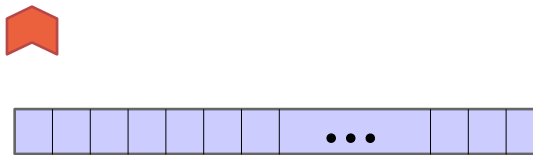
`open(..., O_RDONLY)`

`open(..., O_WRONLY|O_CREAT|O_TRUNC)`

`open(..., O_WRONLY|O_CREAT|O_APPEND)`

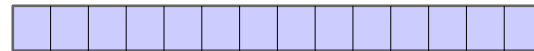


lecture/écriture sur le premier octet



écriture après le dernier octet / lecture impossible

0 1 2 3 4 5 6 7 8 9 10 11 12 13



contrôle de la position courante

`lseek(fd,0,SEEK_SET)`
= ?



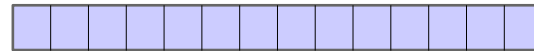
`lseek(fd,0,SEEK_CUR)`
= ?



`lseek(fd,0,SEEK_END)`
= ?

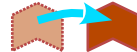


0 1 2 3 4 5 6 7 8 9 10 11 12 13



contrôle de la position courante avec offset

`lseek(fd,2,SEEK_SET)`
= ?



`lseek(fd,-2,SEEK_CUR)`
= ?



`lseek(fd,-2,SEEK_END)`
= ?



Primitives d'Entrées-Sorties : Contrôle de la position courante

- Exercice 3.2

Retour à une position mémorisée

On suppose que le descripteur `fd` est associé à un fichier ouvert en lecture

1. écrire l'instruction permettant de mémoriser dans une variable `position` la position courante dans le fichier
2. écrire l'instruction permettant de revenir à une position mémorisée dans la variable `position`.

Primitives d'Entrées-Sorties : Contrôle de la position courante

- Exercice 3.3

Détermination de la taille d'un fichier

écrire l'instruction permettant de récupérer dans une variable `taille` la taille (nombre d'octets) d'un fichier ouvert en lecture et de descripteur `fd`.

- Exercice 3.4

Accès direct au contenu d'un fichier binaire

On suppose que le descripteur `fd` est associé à un fichier binaire ouvert en lecture et contenant des `int`.

écrire une fonction qui retourne le $n^{\text{ième}}$ `int` de ce fichier.

Primitives d'Entrées-Sorties : Retour sur l'ouverture d'un fichier

- Dans la fonction `int open (const char *désignation, int mode [, mode_t droits] )` le paramètre `mode` indique comment on accède aux données du fichier.

Notamment :

- `O_APPEND` interdit d'écraser ce qui a déjà été écrit pendant toute la session (jusqu'à la fermeture du fichier avec `close`). Même si on revient en arrière, avec `lseek` par exemple, l'écriture se fera toujours à la fin du fichier (*`lseek` n'agit alors que sur la lecture*).
 - Si on utilise `O_TRUNC` en combinaison avec `O_APPEND`, le contenu du fichier est d'abord écrasé (`O_TRUNC`) mais par la suite on ne pourra pas écraser les données nouvellement écrites. L'écriture se fera à la fin (`O_APPEND`), durant toute la session.
 - Avec `O_SYNC` le système synchronise le tampon sur le disque à chaque nouvelle écriture même si le bloc n'est pas plein, quitte à réécrire plusieurs fois le même bloc...
- Exemple : les fichiers de log...

Primitives d'Entrées-Sorties : Duplication de descripteur

Chaque processus possède une table de descripteurs de fichiers. Il est possible de dupliquer un descripteur de fichier. Les deux descripteurs désigneront alors le même fichier. Ils pourront être utilisés de manière interchangeable.

```
#include <unistd.h>
```

```
int dup(int fd)
```

- duplique le descripteur `fd` et utilise le premier descripteur libre de la table comme nouveau descripteur

```
int dup2(int fdsource, int fddestination)
```

- duplique le descripteur `fdsource` dans le descripteur `fddestination` après avoir fermé le fichier associé si nécessaire.
- ces fonctions retournent le nouveau descripteur ou `-1` en cas de problème

Primitives d'Entrées-Sorties : Duplication de descripteur

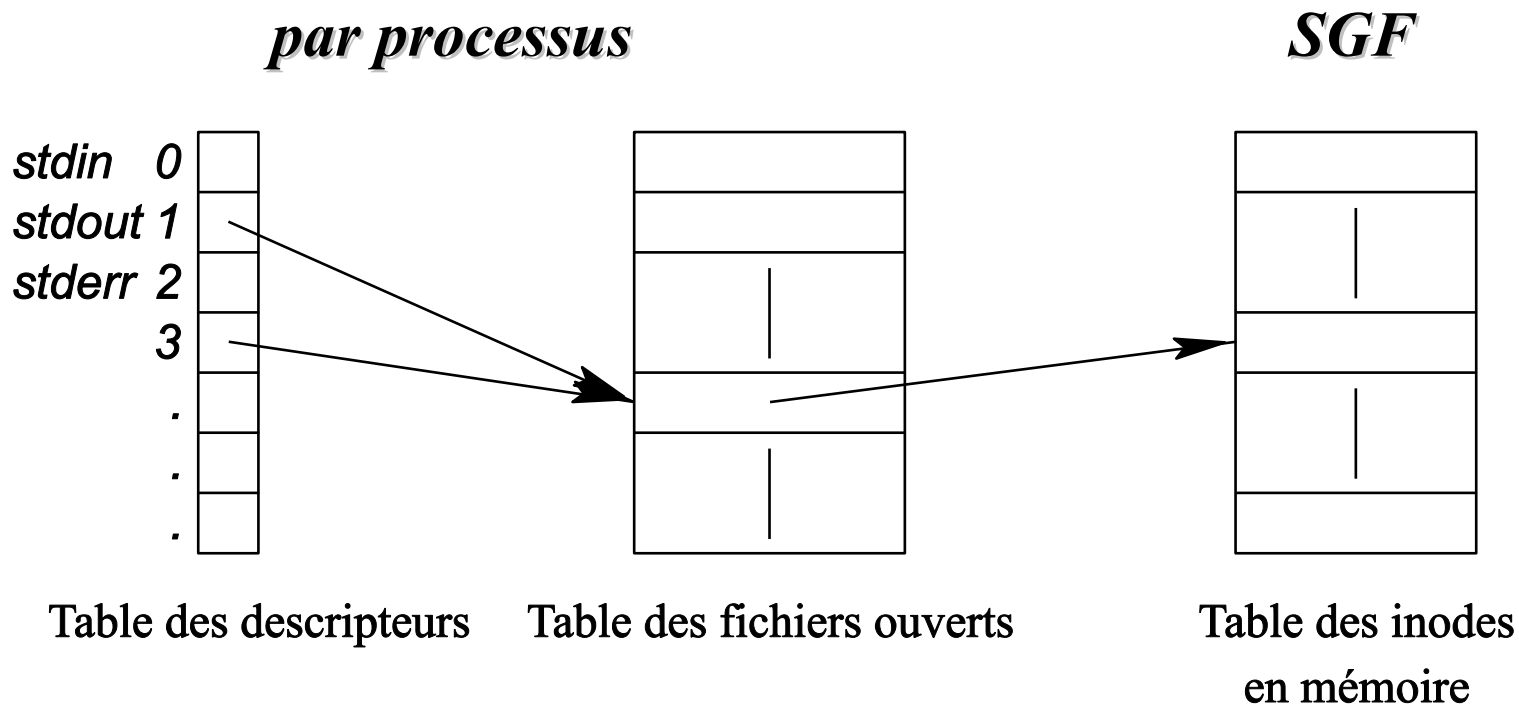
Exemple : redirection de la sortie standard vers un fichier.

```
int d;  
d=open("fich", O_WRONLY|O_CREAT|O_TRUNC, S_IRUSR|S_IWUSR);  
if (d==-1)  
    perror("fich");  
else  
{  
    printf("Bonjour\n");  
    dup2(d,1);      /* ou bien dup2(d, STDOUT_FILENO);    ou bien close(1); dup(d); */  
    printf("Bonsoir\n");  
}
```

Lors de l'exécution de cet extrait de programme, **Bonjour** est affiché à l'écran et **Bonsoir** est écrit dans le fichier **fich**.

Primitives d'Entrées-Sorties : Duplication de descripteur

Exemple : redirection de la sortie standard vers un fichier



Primitives d'Entrées-Sorties : Duplication de descripteur

- Exercice 3.5

écrire un programme C qui lance l'exécution d'une commande en réalisant les redirections du type : `commande < fich_entree >> fich_sortie`

- Il prend trois arguments : la commande, la désignation du fichier vers lequel doit être redirigée l'entrée standard et la désignation du fichier vers lequel doit être redirigée la sortie standard (avec ajout en fin de fichier).
- L'exécution de la commande doit être déléguée à un processus *Fils*.
- Les redirections doivent être réalisées dans le *Père* par duplication des descripteurs de fichiers. Si le fichier de sortie n'existe pas, il doit être créé avec le droit en lecture pour tous et le droit en écriture pour le propriétaire.
- Le programme doit gérer les codes de retour en cas d'erreur (mauvais nombre de paramètres, fichier inaccessible, etc.).

Primitives d'Entrées-Sorties : Suppression d'un fichier

```
#include <unistd.h>
```

```
int unlink(char *désignation)
```

- supprime le lien physique vers le fichier désignation .
- retourne 0 en cas de succès ou -1 en cas d'échec.
- La fonction `remove` de la bibliothèque standard du C, fait appel à `unlink` sous Unix.

Remarque : supprimer un fichier n'efface pas ses données sur le disque. Elles restent présentes sur le support tant que les blocs qu'elles occupent ne sont pas réutilisés pour d'autres données.

Quoi qu'il en soit, si vous avez des données sensibles → cryptez-les !

Primitives d'Entrées-Sorties : Tronquer un fichier

```
#include <unistd.h>
```

```
int ftruncate(int fd, off_t taille);
```

- change la taille (*tronque*) du fichier de descripteur `fd` à `taille` octets.
le fichier doit être ouvert en écriture.
- si la taille est réduite, les données existant au-delà sont perdues
- si la taille est augmentée, selon le SE et le Système de Fichiers* :
soit des zéros sont ajoutés, soit il y a une erreur...
- retourne 0 en cas de succès ou -1 en cas d'échec.

existe aussi : `int truncate(const char * désignation, off_t taille);`

* : par exemple sous Linux, `ftruncate` permet d'augmenter la taille avec les Systèmes de Fichiers natifs mais pas VFAT...

Primitives d'Entrées-Sorties & Multi-processus

- Rappel : après un fork, le processus fils hérite d'une copie des descripteurs de fichiers déjà ouverts dans le processus père
 - Problème : accès concurrent à ces fichiers.
 - ◆ affichages entrelacés sur stdout par exemple
 - Mais il y a plus subtil : après appel à read, write ou lseek, la position courante est modifiée pour tous les processus utilisant le même descripteur de fichier (le père et ses fils créés après l'ouverture du fichier).
- Utiliser lseek avant read ou write pour être au bon endroit dans le fichier ?
- Déconseillé dans ce cas : l'ordonnanceur peut préempter le processus entre les deux appels, ce qui rend la position pour la lecture ou l'écriture imprévisible

Primitives d'Entrées-Sorties & Multi-processus

→ utiliser des primitives qui réalisent dans le même appel à la fois le déplacement dans le fichier et la lecture ou l'écriture :

```
ssize_t pread(int fd, void * buf, size_t count, off_t offset);
```

```
ssize_t pwrite(int fd, const void * buf, size_t count, off_t offset);
```

- fonctionnent comme `read` et `write`, en déplaçant au préalable la position courante à `offset` octets par rapport au début du fichier.

opération atomique : ne peut être interrompue par l'ordonnanceur

→ le déplacement dans le fichier est immédiatement suivi d'une lecture ou écriture

Fichier Binaire vs Fichier Texte

- Même si, au bout du compte, tout est binaire, on distingue généralement deux types de fichiers :
 - ◆ les fichiers de texte : chaque octet qui les compose représente un caractère et ces caractères sont organisés en lignes (le contenu apparaît de manière lisible dans un éditeur de texte) ;
 - ◆ les fichiers binaires : tous les autres fichiers (leur contenu apparaît sous la forme de caractères illisibles dans un éditeur de texte).
- La différence entre les deux types de fichiers réside dans la manière dont les informations y sont codées. *Exemple*, l'entier 24 est codé par :
 - ◆ 00110010 00110100 dans un fichier de texte : un octet pour le code ASCII du caractère 2, qui vaut 50, et un octet pour le code ASCII du caractère 4, qui vaut 52 ;
 - ◆ 00000000 00000000 00000000 00011000 dans un fichier binaire si les entiers sont codés sur quatre octets.